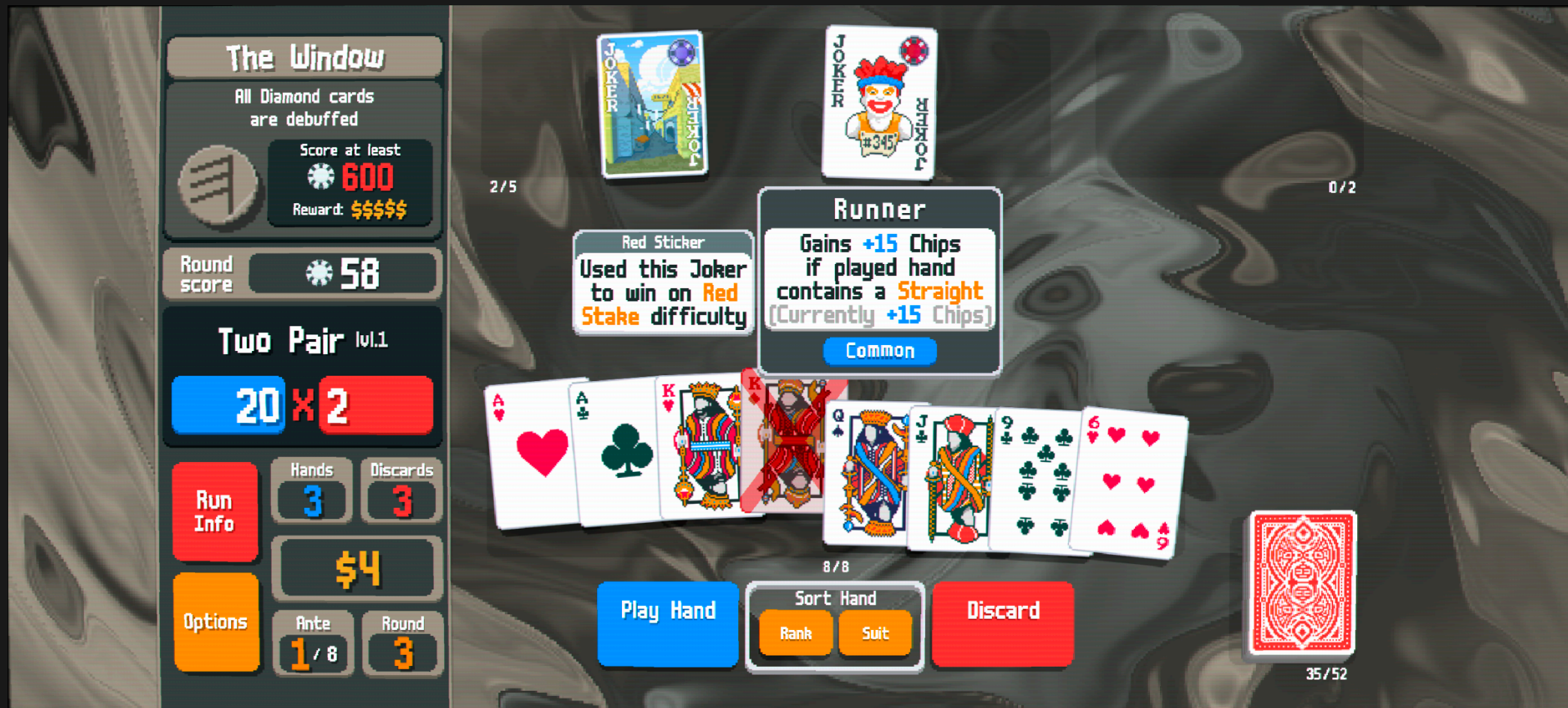


Brute Force, Binomials and Balatro

Paul Stanford · Permutive · London Scala User Group

What is Balatro?



seed 94IKWPES, if you want to play it yourself

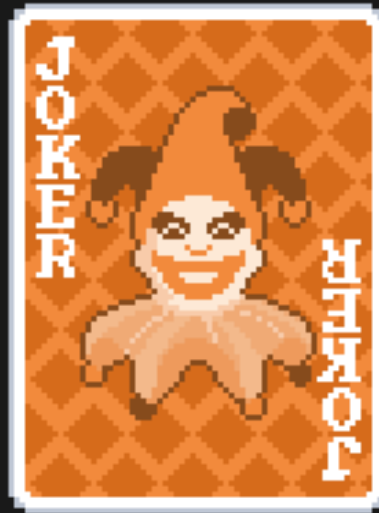
Jokers

Joker



+4 Mult

Greedy Joker



Played cards with Diamond suit
give +3 Mult when scored

Crazy Joker



+12 Mult if played hand contains
a Straight

Runner



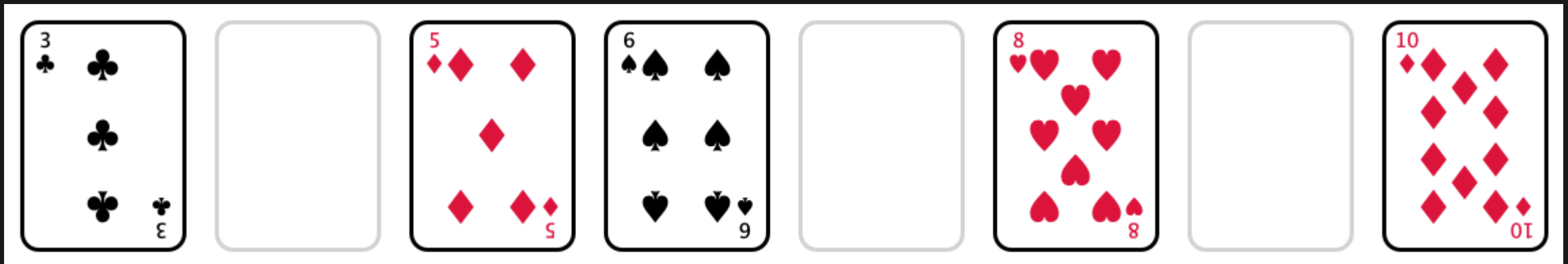
Gains +15 Chips if played hand
contains a Straight

All of these jokers just help increase the score

Shortcut Joker



Allows Straights to be made with **gaps of 1 rank** (ex: 10 8 6 5 3)



This Joker fundamentally changes the rules!

The Question

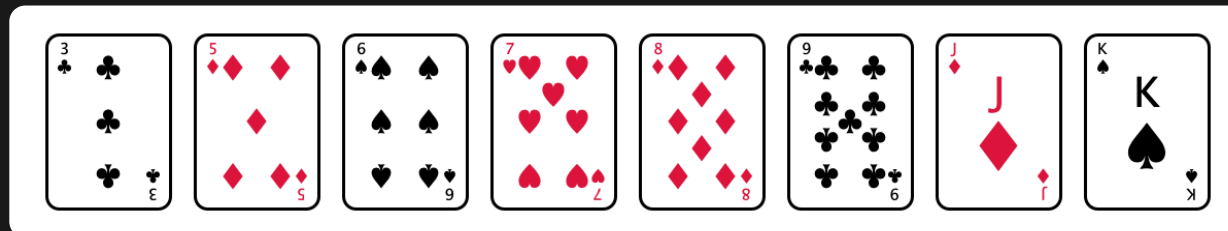
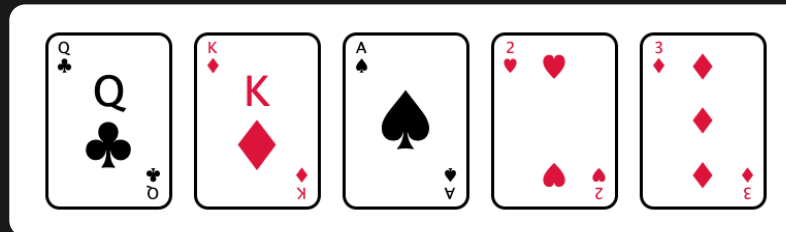
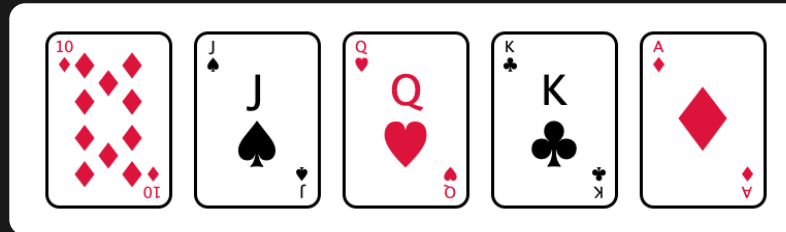
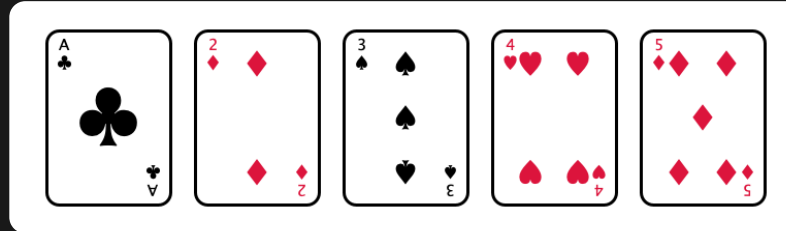
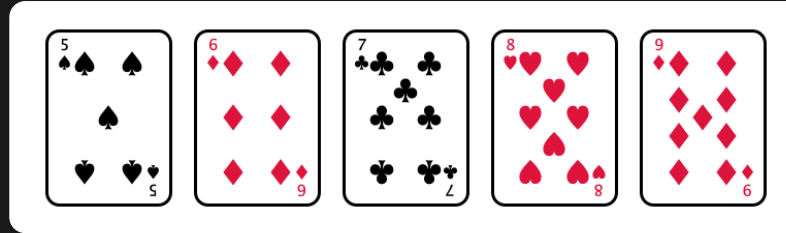
Playing with the Shortcut Joker I felt like I was getting loads of straights easily! But can this be quantified?

Given a standard 52 card deck what is the probability that the first 8 cards I draw contain a straight where gaps of one are allowed?

But first, a simpler question

Given a standard 52 card deck what is the probability that the first 8 cards I draw contain a **normal** straight?

What is a Straight?



Brute Force

```
enum Rank:
  case Ace, Two, Three, Four, Five, Six, Seven,
    Eight, Nine, Ten, Jack, Queen, King
enum Suit:
  case Spade, Heart, Club, Diamond
case class Card(rank: Rank, suit: Suit)

val fullDeck: List[Card] =
  for
    rank <- Rank.values.toList
    suit <- Suit.values.toList
  yield Card(rank, suit)

def containsStraight(hand: List[Card]): Boolean = ???

fullDeck.combinations(8).count(containsStraight)
```

You just do it, and it works: 73,870,336 of 752,538,150 hands, 9.82%

1. But it tells me nothing about the problem. I still don't know *why* it's 9.82%
2. And it took 3 min 38 s

~~Brute Force,~~ Binomials and Balatro

Paul Stanford · Permutive · London Scala User Group



Mathematical Solution

1. Simplest case: a hand of 5 cards
2. More simple cases, to build intuition
3. The general case
4. The general case, in Scala
5. Introduce gaps

5 cards

First we can work out how many straights exist if we ignore suits

To do this we need only consider the **rank sets** of a hand

I define a "rank set" as the set of distinct ranks in a hand

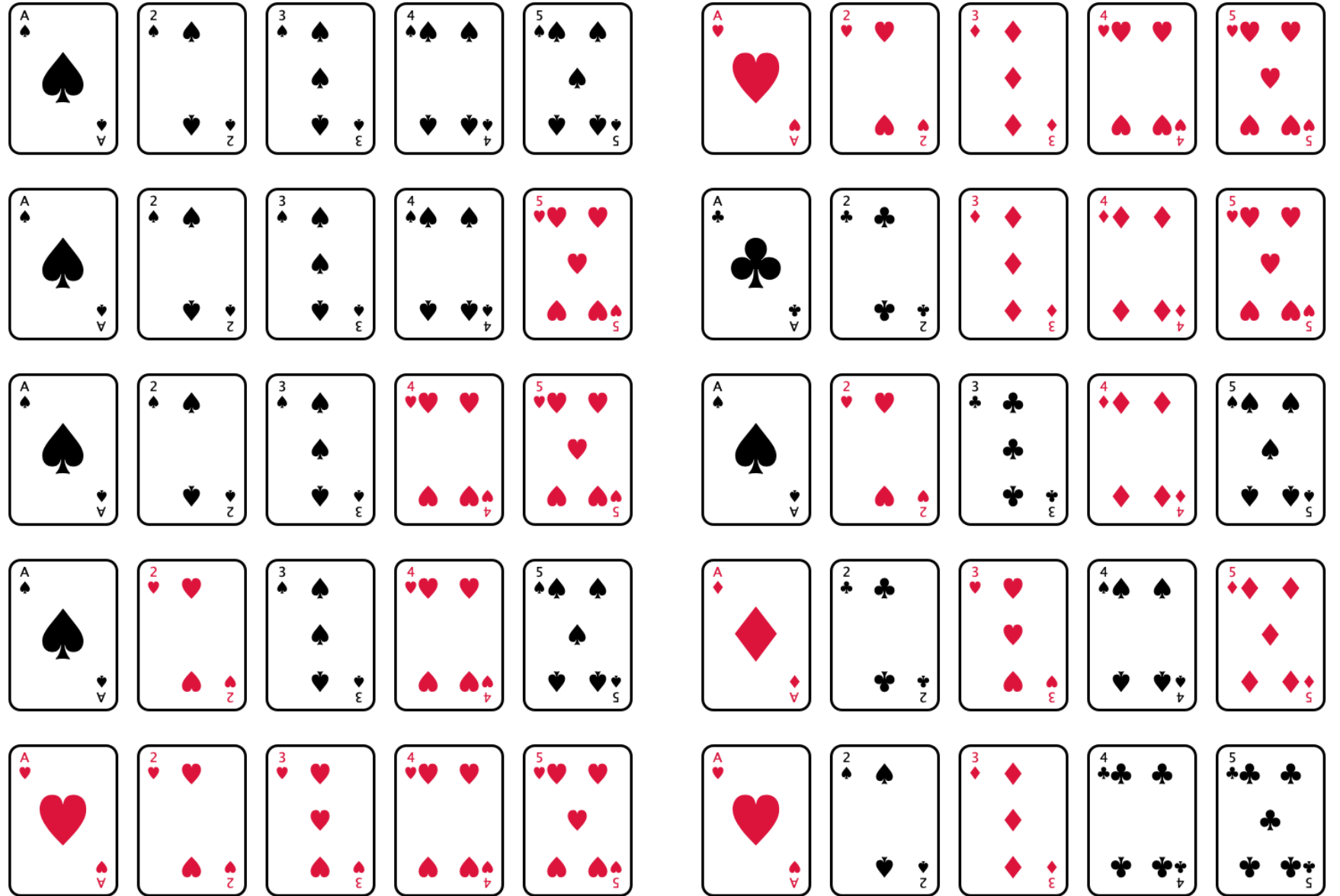
A	2	3	4	5	6	7	8	9	10
2	3	4	5	6	7	8	9	10	J
3	4	5	6	7	8	9	10	J	Q
4	5	6	7	8	9	10	J	Q	K
5	6	7	8	9	10	J	Q	K	A

5 cards

For each card we have 4 choices of suit, and there are 5 cards

$$4^5 = 1,024$$

This is just a handful of examples with the first rank set, A to 5. Well, a few handfuls I guess!



5 cards

All 1,024 ways to
choose suits for
one rank set

- ♠ saffron
- ♥ hot coral
- ♣ cerulean
- ♦ deep purple



5 cards

$$S = 10$$

the number of rank sets that give a straight

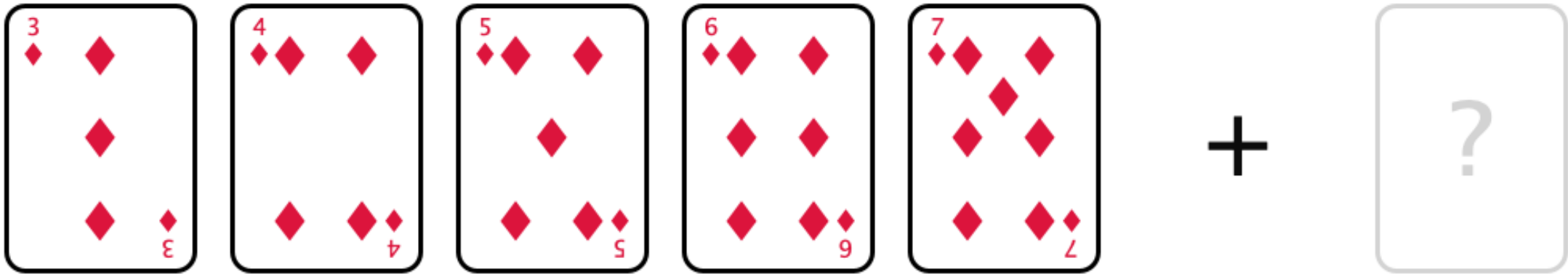
$$Su = 4^5 = 1,024$$

the **suit factor**: the number of ways to choose suits for a given rank set

$$S \times Su = 10,240$$

Six-Card Hands

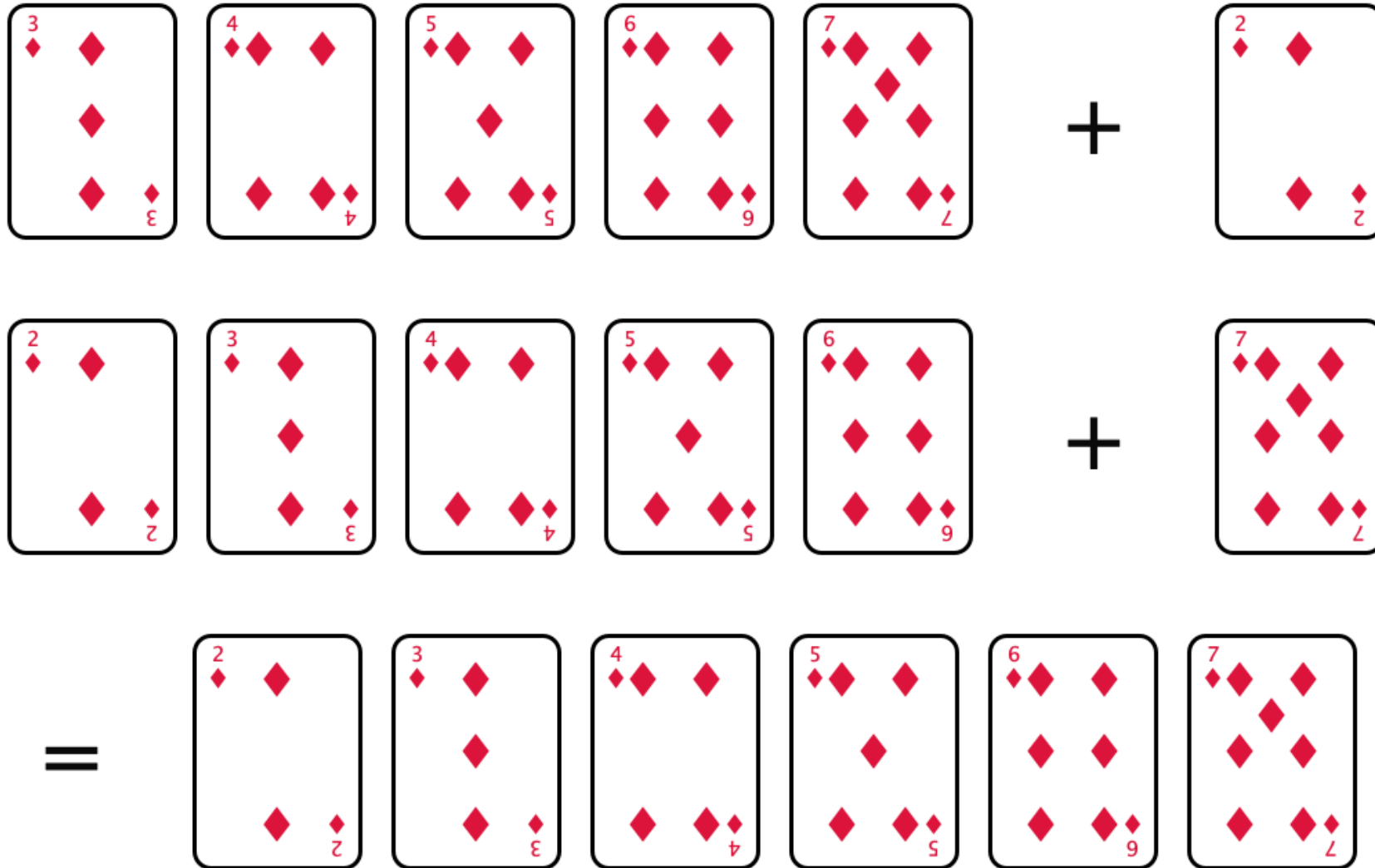
How many hands of 6 cards contain a straight?



10,240 5-card straights multiplied by 47 choices of sixth card = 481,280

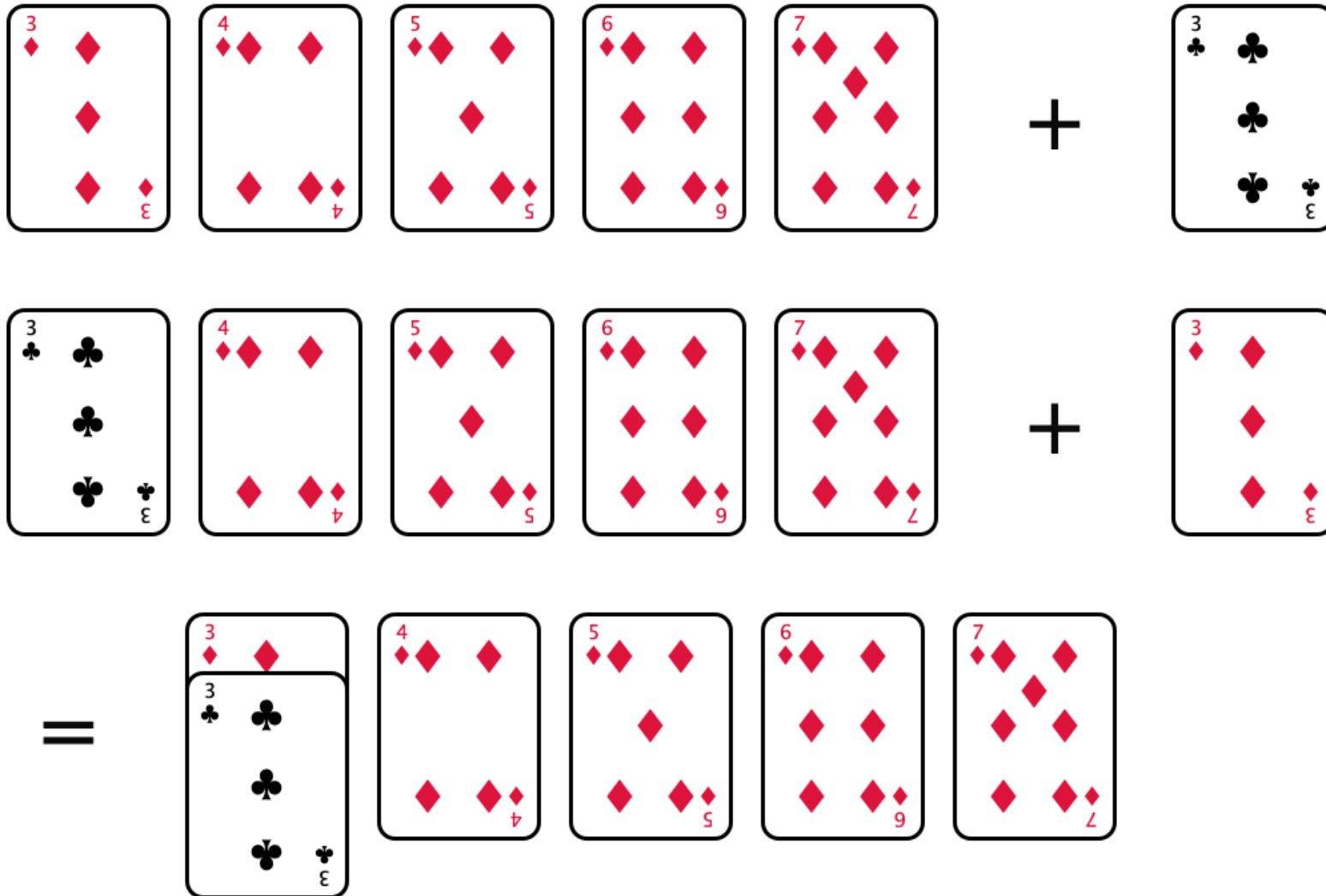
But brute force says 367,616. So what did I do wrong?

Double Counting Demons



If I have a run of 6 cards
there's always going to be
2 ways to get to it!

Devious Double Counting Demons

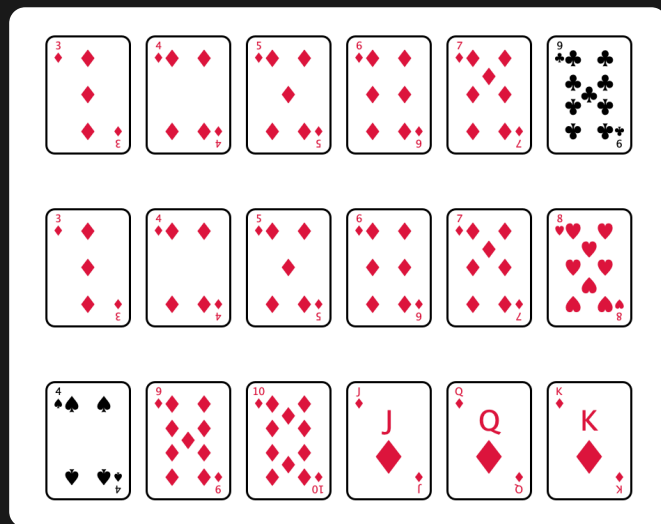


More subtly if I have a hand with a straight and a duplicate rank there's also going to be two ways to get to it!

Two Classes of Demon

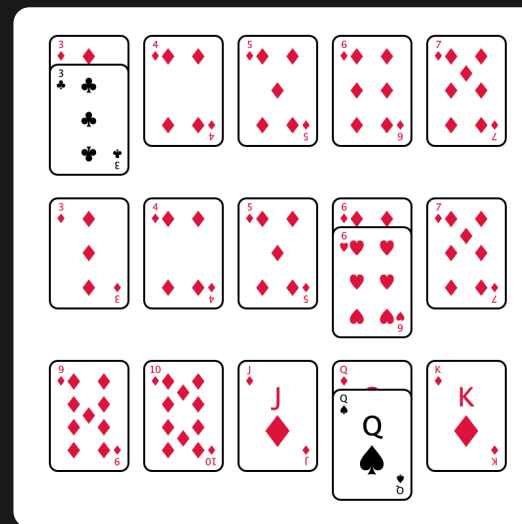
There are two different types of double counting which correspond to different "templates" for the hand; the difference is the number of **distinct ranks** in each hand

Each template can be thought of as a **multiset**



six distinct ranks

$\{1, 1, 1, 1, 1, 1\}$



five distinct ranks, one pair

$\{2, 1, 1, 1, 1\}$

Multisets

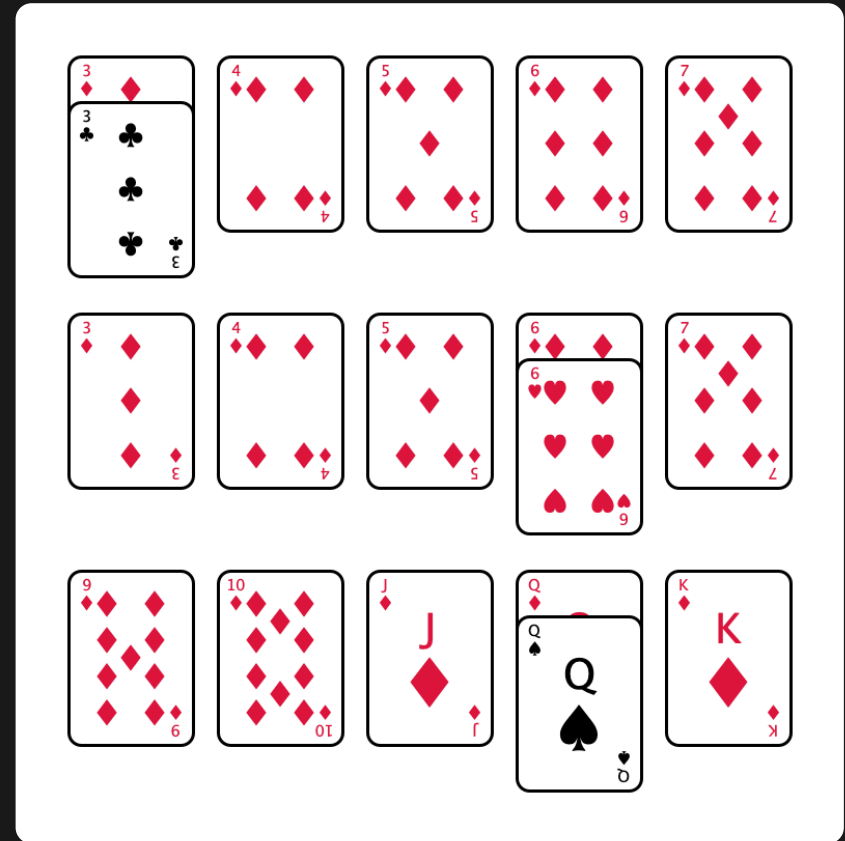
A set is an unordered collection of objects where each object can appear at most once

A **multiset** is like a set but each element can appear multiple times; each element has a **multiplicity**. In Scala terms it is a bit like a `Map[A, Int]`, from each element to its multiplicity

We can denote a multiset more compactly by using exponents to count how many times each element appears: $\{2, 1, 1, 1, 1\} = \{2, 1^4\}$ and $\{1, 1, 1, 1, 1, 1\} = \{1^6\}$

Each multiset corresponds to a template for our hand; there are many possible ways to fill in this template by choosing different suits and choosing which ranks get paired up (or later tripled up)

I call these templates **degeneracy classes** because different templates have different repeated (or degenerate) ranks

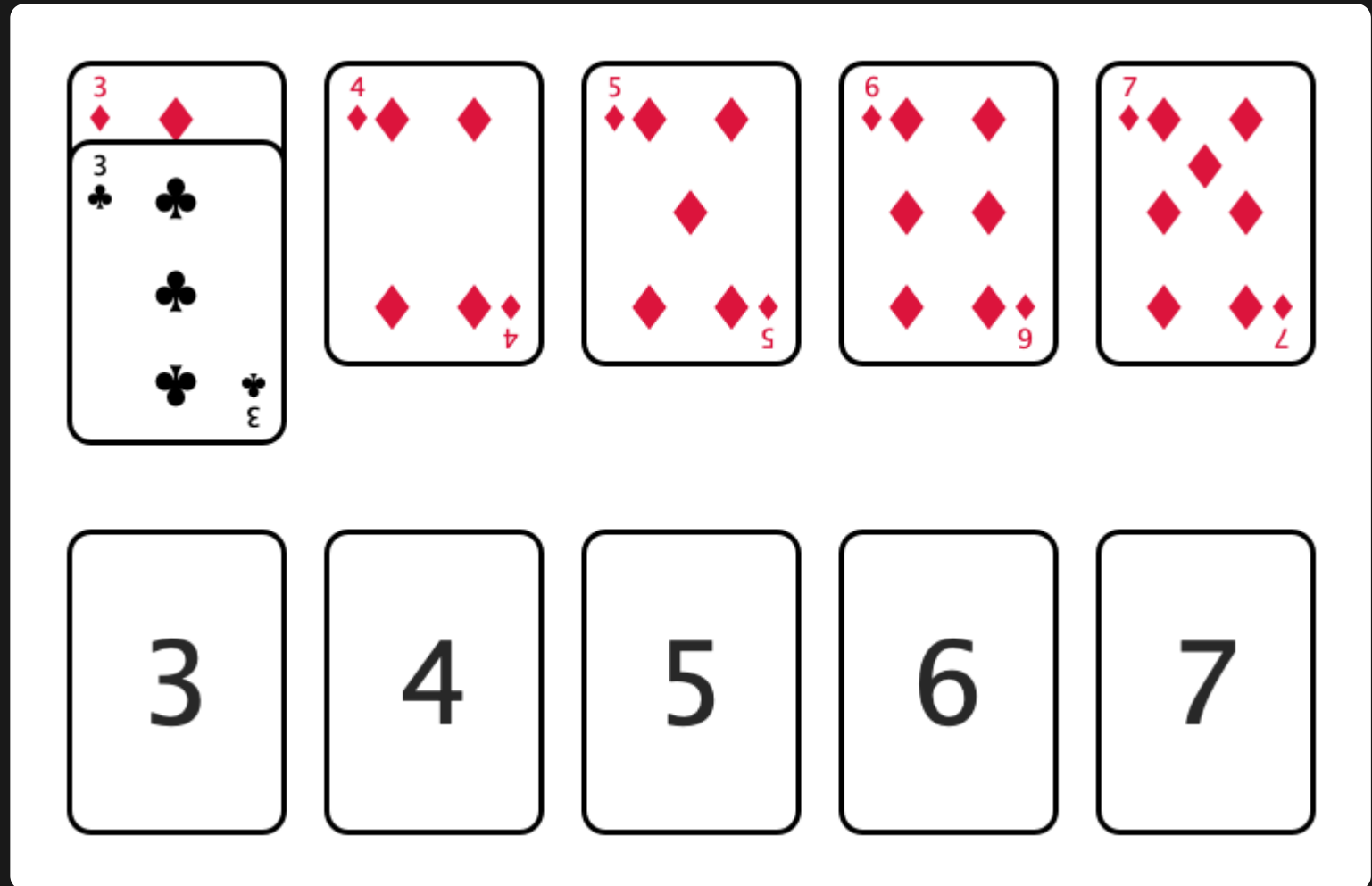


One Pair: Rank Sets

$$\{2, 1^4\}$$

We only care about the number of rank sets that contain a straight. As with the 5 card case we only have 5 distinct ranks here so we have $S = 10$.

Write it $S(r)$, with r the number of distinct ranks: $S(5) = 10$



One Pair: Placements

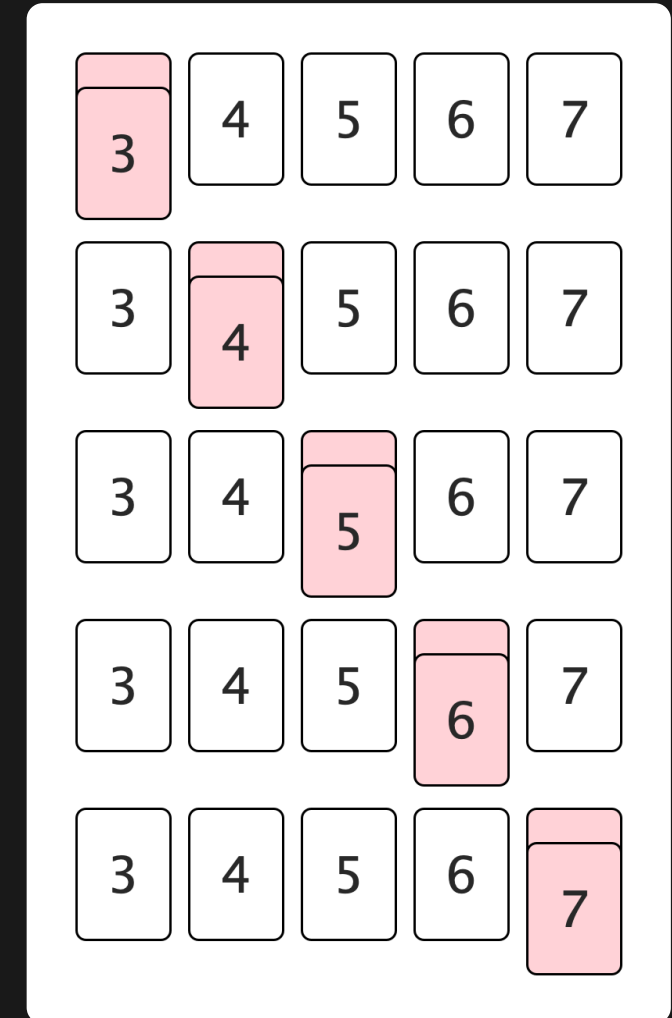
$\{2, 1^4\}$

A **placement** says which rank plays which role (i.e. which rank gets to be a pair, which gets to be a triple, etc). We can let $P(d)$ denote the total placements for a degeneracy class d .

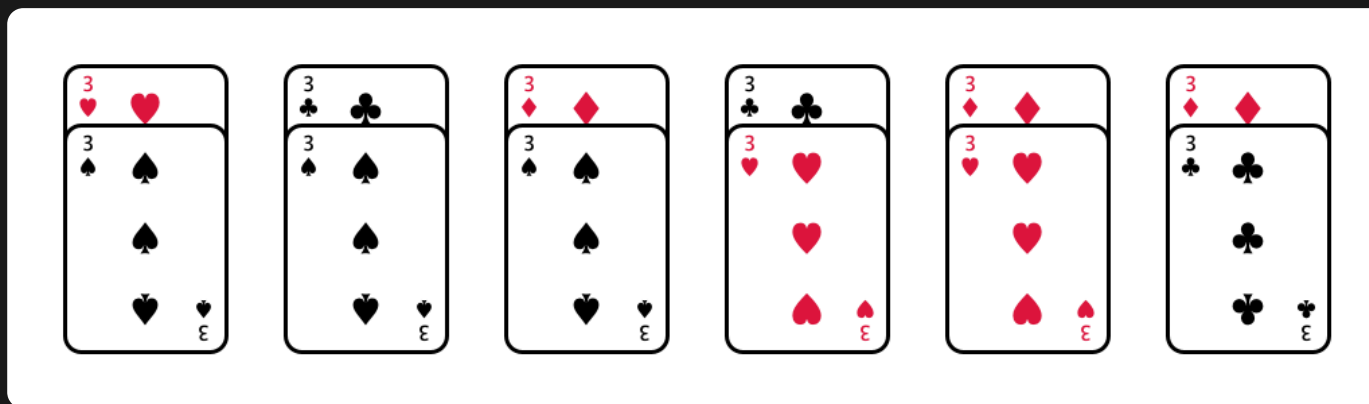
Which rank gets the pair? 5 choices: $P = 5$

Or: the distinct orderings of $\{2, 1, 1, 1, 1\}$ along the five ranks. 2 1 1 1 1, 1 2 1 1 1, ... five of them

■ the pair



One Pair: Suits



Which 2 of the 4 suits does the pair get? 6 ways

Written $\binom{4}{2}$, "**4 choose 2**": the number of ways to choose 2 things from 4

Each single picks 1 suit of 4: $\binom{4}{1} = 4$ ways

$$Su = \binom{4}{2} \binom{4}{1}^4 = 6 \times 256 = 1,536$$

$$S(5) \times P \times Su = 10 \times 5 \times 1,536 = 76,800$$

Binomial Coefficients

♠	♥	♣	♦
✓	✓	✗	✗
✓	✗	✓	✗
✓	✗	✗	✓
✗	✓	✓	✗
✗	✓	✗	✓
✗	✗	✓	✓

$\binom{4}{2}$ is the number of distinct orderings of $\{\checkmark, \checkmark, \times, \times\}$ along the four suits

Four different things line up in $4! = 4 \times 3 \times 2 \times 1 = 24$ ways

Swapping the two $\checkmark$ s, or the two $\times$ s, changes nothing: divide by $2!$ twice

$$\binom{4}{2} = \frac{4!}{2! 2!} = \frac{24}{4} = 6$$

In general $\binom{n}{k} = \frac{n!}{k! (n - k)!}$

$(x + y)^4 = x^4 + 4x^3y + 6x^2y^2 + 4xy^3 + y^4$: the 6 is the orderings of $x x y y$

And $\binom{52}{8} = 752,538,150$: every 8-card hand.

Six Distinct Ranks

$\{1^6\}$, no duplicates

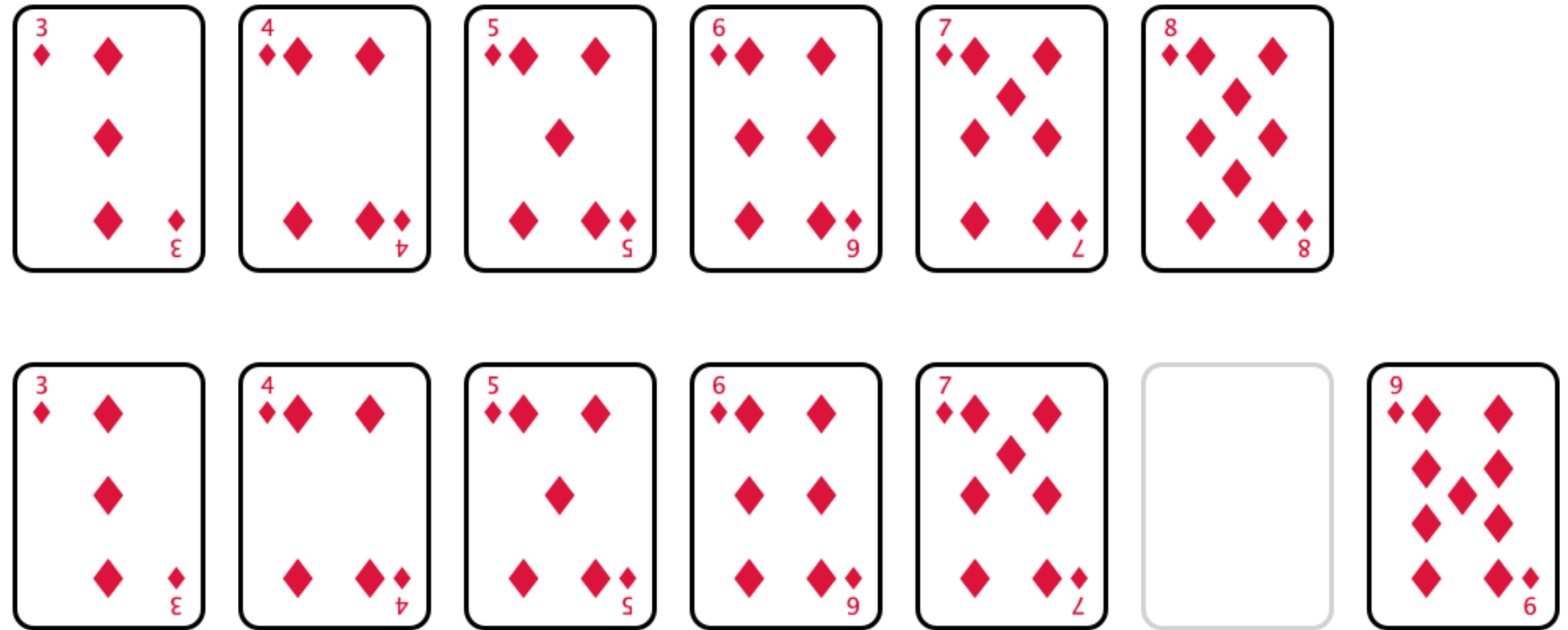
Either a run of six ranks: 9
of them

Or a 5-card straight plus a
rank that doesn't "touch"
it: 62

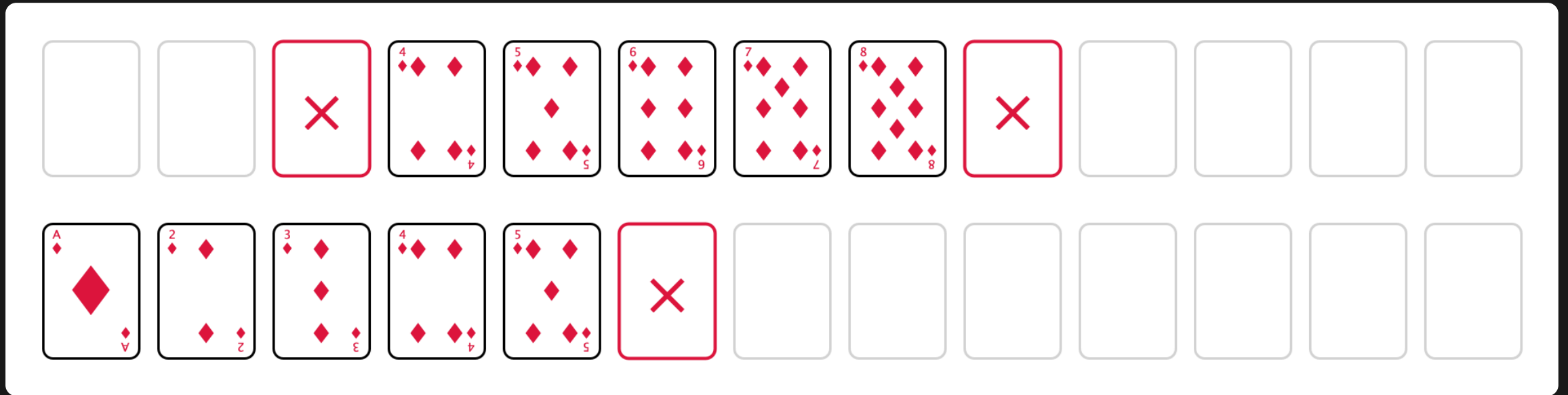
$$S(6) = 9 + 62 = 71$$

$$P = 1, Su = \binom{4}{1}^6$$

$$71 \times 4,096 = 290,816$$



Where the 62 Comes From



A 5-card straight, plus a sixth rank that doesn't "touch" it

8 straights sit in the middle: $13 - 5 - 2 = 6$ free ranks each

2 straights end on an Ace: $13 - 5 - 1 = 7$ free ranks each

$$8 \times 6 + 2 \times 7 = 62$$

All 71 of Them

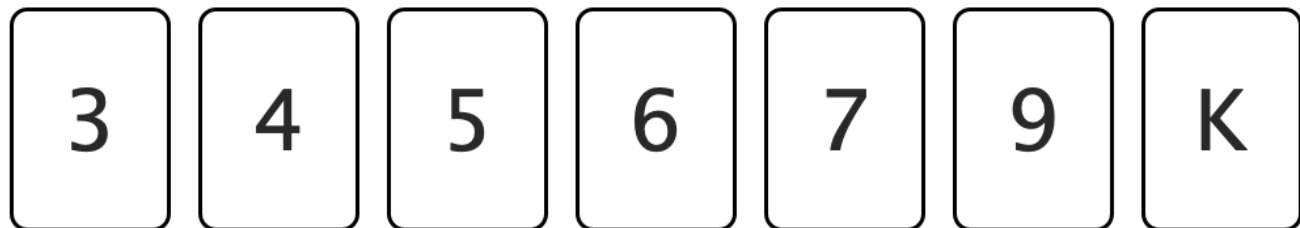
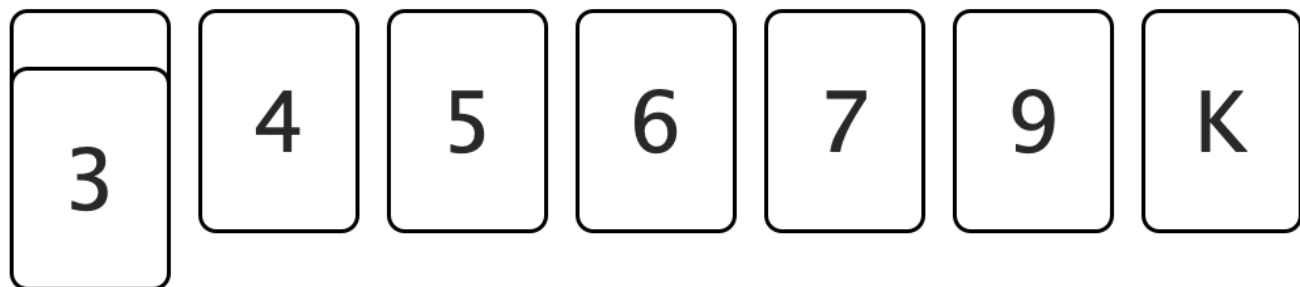
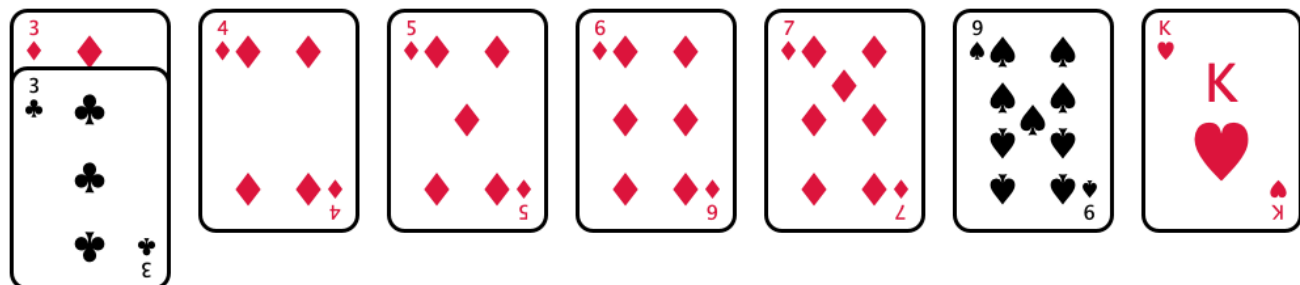
A	2	3	4	5	6	A	2	3	4	5	7	A	2	3	4	5	8	A	2	3	4	5	9	A	2	3	4	5	10	A	2	3	4	5	J	A	2	3	4	5	Q	A	2	3	4	5	K											
2	3	4	5	6	7	2	3	4	5	6	8	2	3	4	5	6	9	2	3	4	5	6	10	2	3	4	5	6	J	2	3	4	5	6	Q	2	3	4	5	6	K																	
3	4	5	6	7	8	A	3	4	5	6	7	3	4	5	6	7	9	3	4	5	6	7	10	3	4	5	6	7	J	3	4	5	6	7	Q	3	4	5	6	7	K																	
4	5	6	7	8	9	A	4	5	6	7	8	2	4	5	6	7	8	4	5	6	7	8	10	4	5	6	7	8	J	4	5	6	7	8	Q	4	5	6	7	8	K																	
5	6	7	8	9	10	A	5	6	7	8	9	2	5	6	7	8	9	3	5	6	7	8	9	5	6	7	8	9	J	5	6	7	8	9	Q	5	6	7	8	9	K																	
6	7	8	9	10	J	A	6	7	8	9	10	2	6	7	8	9	10	3	6	7	8	9	10	4	6	7	8	9	10	4	6	7	8	9	10	Q	6	7	8	9	10	K																
7	8	9	10	J	Q	A	7	8	9	10	J	2	7	8	9	10	J	3	7	8	9	10	J	4	7	8	9	10	J	4	7	8	9	10	J	5	7	8	9	10	J	7	8	9	10	J	K											
8	9	10	J	Q	K	A	8	9	10	J	Q	2	8	9	10	J	Q	3	8	9	10	J	Q	4	8	9	10	J	Q	4	8	9	10	J	Q	5	8	9	10	J	Q	6	8	9	10	J	Q											
9	10	J	Q	K	A	2	9	10	J	Q	K	3	9	10	J	Q	K	4	9	10	J	Q	K	5	9	10	J	Q	K	5	9	10	J	Q	K	6	9	10	J	Q	K	7	9	10	J	Q	K											
						2	10	J	Q	K	A	3	10	J	Q	K	A	4	10	J	Q	K	A	5	10	J	Q	K	A	5	10	J	Q	K	A	6	10	J	Q	K	A	7	10	J	Q	K	A	8	10	J	Q	K	A					

Six Cards, Counted

class	multiset	r	$S(r)$	$P(d)$	$Su(d)$	product
no duplicates	$\{1^6\}$	6	71	$\binom{6}{6} = 1$	$\binom{4}{1}^6 = 4,096$	290,816
one pair	$\{2^1, 1^4\}$	5	10	$\binom{5}{1} = 5$	$\binom{4}{2} \binom{4}{1}^4 = 1,536$	76,800
					sum	367,616

Every hand is now built exactly once, the degeneracy classes can't ever overlap

The Insight



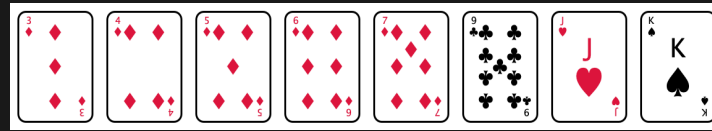
Whether or not a hand contains a straight depends only on the **set** of ranks in that hand

Everything else, the suits and the placements, is just counting how many different hands share that rank set

That's why $S(r)$ needs nothing but r

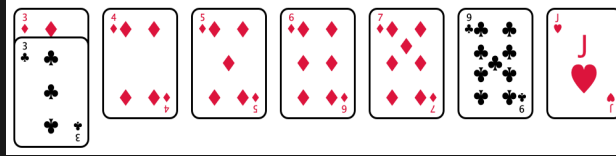
And there are only $2^{13} = 8,192$ possible rank sets since each rank either appears or it doesn't. This is far fewer than the total number of hands

Degeneracy Classes for Eight Cards



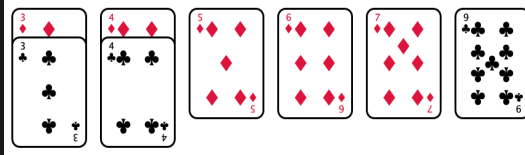
$$\{1^8\}$$

no duplicates



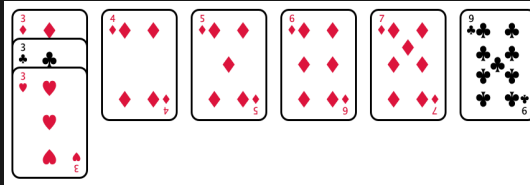
$$\{2, 1^6\}$$

one pair



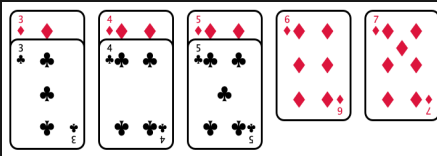
$$\{2^2, 1^4\}$$

two pairs



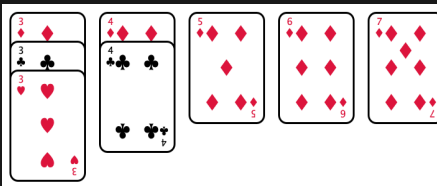
$$\{3, 1^5\}$$

three of a kind



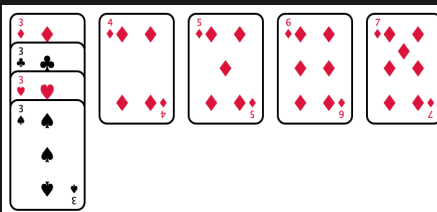
$$\{2^3, 1^2\}$$

three pairs



$$\{3, 2, 1^3\}$$

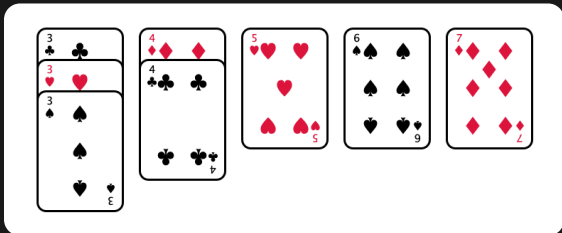
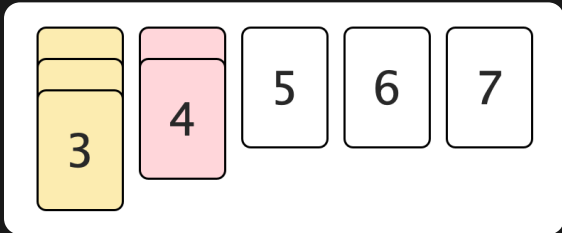
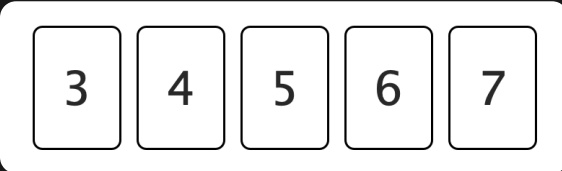
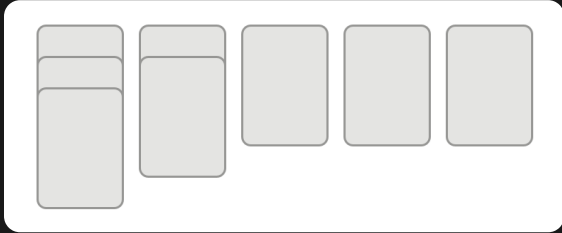
full house



$$\{4, 1^4\}$$

four of a kind

One Hand, Four Choices



1. Choose a degeneracy class: (e.g. $\{3, 2, 1^3\}$)

2. Choose a rank set that contains a straight (e.g. 3 to 7)

3. Choose a placement (e.g. 3 gets to be a triple, 4 gets to be a pair)

4. Choose suits for every card

A General Approach

1. Choose a degeneracy class, d
2. Choose a rank set that contains a straight: $S(r)$ ways
3. Place the card counts on those ranks: $P(d)$ ways
4. Choose the suits: $Su(d)$ ways
5. Every hand comes from exactly one chain of choices, and since the choices after fixing the degeneracy class are independent we can just multiply them! $S(r) \times P(d) \times Su(d)$
6. Sum over the degeneracy classes

$$\text{hands with a straight} = \sum_d S(r) P(d) Su(d)$$

Maths or Code?

We can derive a formula

- $P(d)$, the placements
- $Su(d)$, the suits

I was unable to derive a formula so used Scala

- Listing every degeneracy class d for a hand size
- $S(r)$, the rank sets with a straight

Placements

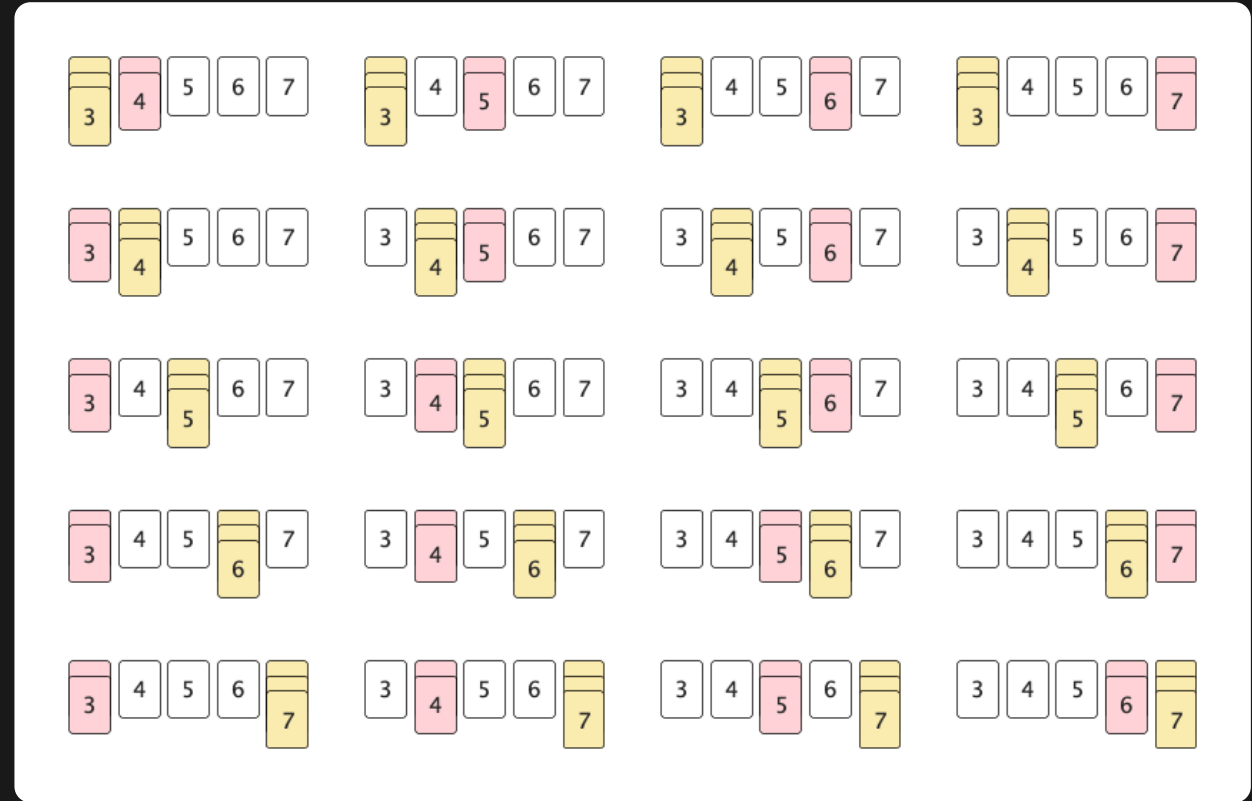
$\{\mathbf{3}, \mathbf{2}, 1^3\}$

■ the triple ■ the pair

$P(d)$ is the number of distinct orderings of $\{\mathbf{3}, \mathbf{2}, 1, 1, 1\}$ along the ranks 3 to 7

$$P(d) = \frac{5!}{\mathbf{1!1!3!}} = \frac{120}{6} = 20$$

Another way to think about this would be to consider which rank gets to be a triple (5 choices) and then which rank gets to be a pair (4 choices.) But we shall see that the "orderings" approach is more generalisable



Placements: a Harder One

$\{\mathbf{3}, \mathbf{2}^2, \mathbf{1}^2\}$

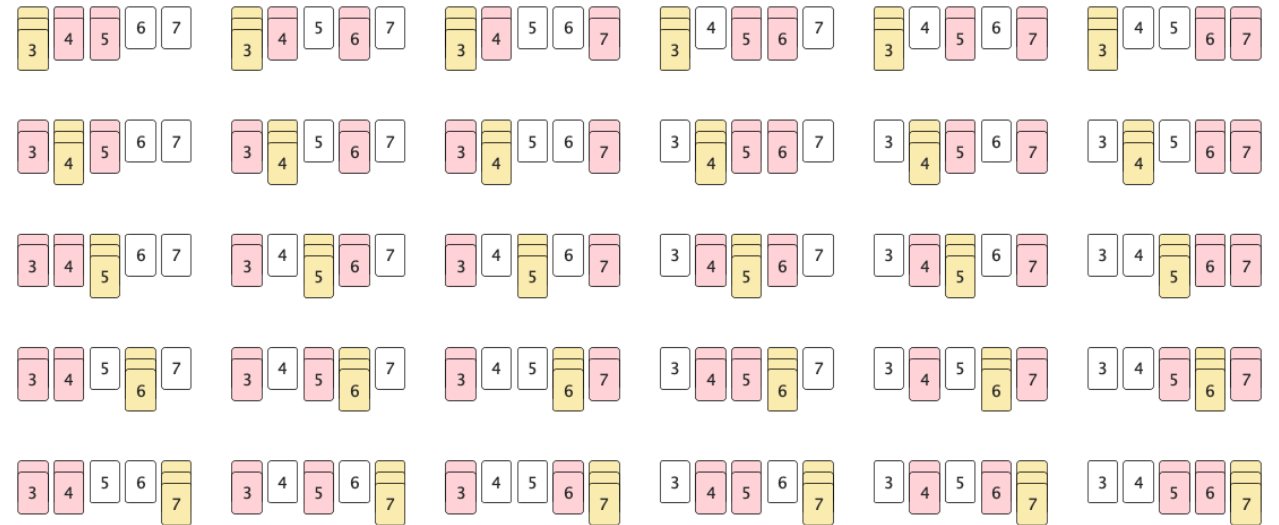
■ the triple ■ the pairs

9 cards: a triple, two pairs and two singles, on the ranks 3 to 7

$P(d)$ is the number of distinct orderings of $\{\mathbf{3}, \mathbf{2}, \mathbf{2}, \mathbf{1}, \mathbf{1}\}$

$$P(d) = \frac{5!}{\mathbf{1!} \mathbf{2!} \mathbf{2!}} = \frac{120}{4} = 30$$

5 choices for the triple, then $\binom{4}{2} = 6$ for the two pairs: $5 \times 6 = 30$



Multinomial Coefficients

3	4	5	6	7
3	2	1 _A	1 _B	1 _C
3	2	1 _A	1 _C	1 _B
3	2	1 _B	1 _A	1 _C
3	2	1 _B	1 _C	1 _A
3	2	1 _C	1 _A	1 _B
3	2	1 _C	1 _B	1 _A

ranks across the top, card counts below:
six line-ups, one placement

Label the singles A, B and C: {3, 2, 1_A, 1_B, 1_C} lines up in $5! = 120$ ways

Shuffling the labels changes nothing: every placement shows up $3! = 6$ times

$$P = \frac{5!}{1! 1! 3!} = \frac{120}{6} = 20$$

That is a **multinomial coefficient**, $\binom{5}{1, 1, 3}$: the distinct orderings of a multiset

$(x + y + z)^5$: the coefficient of $x y z^3$ is $\binom{5}{1, 1, 3} = 20$, the orderings of $x y z z z$

A binomial is just the two-bucket case: $\binom{n}{k} = \binom{n}{k, n-k}$

Multinomial Coefficients: a Harder One

$\{3, 2, 2, 1, 1\}$: a triple, two pairs and two singles

Label the pairs A and B, and the singles A and B:

$\{3, 2_A, 2_B, 1_A, 1_B\}$ lines up in $5! = 120$ ways

Swapping the two pairs changes nothing: divide by $2!$

Swapping the two singles changes nothing either: divide by $2!$ again

$$P = \frac{5!}{1! 2! 2!} = \frac{120}{4} = 30 = \binom{5}{1, 2, 2}$$

$(x + y + z)^5$: the coefficient of $x y^2 z^2$ is 30, the orderings of
 $x y y z z$

Multinomial Coefficients Formula

$$\binom{n}{k_1, k_2, \dots, k_m} = \frac{n!}{k_1! k_2! \cdots k_m!}$$

The number of distinct orderings of a multiset of n elements, where k_i is the multiplicity of the i th element

$$\text{so } n = k_1 + k_2 + \cdots + k_m$$

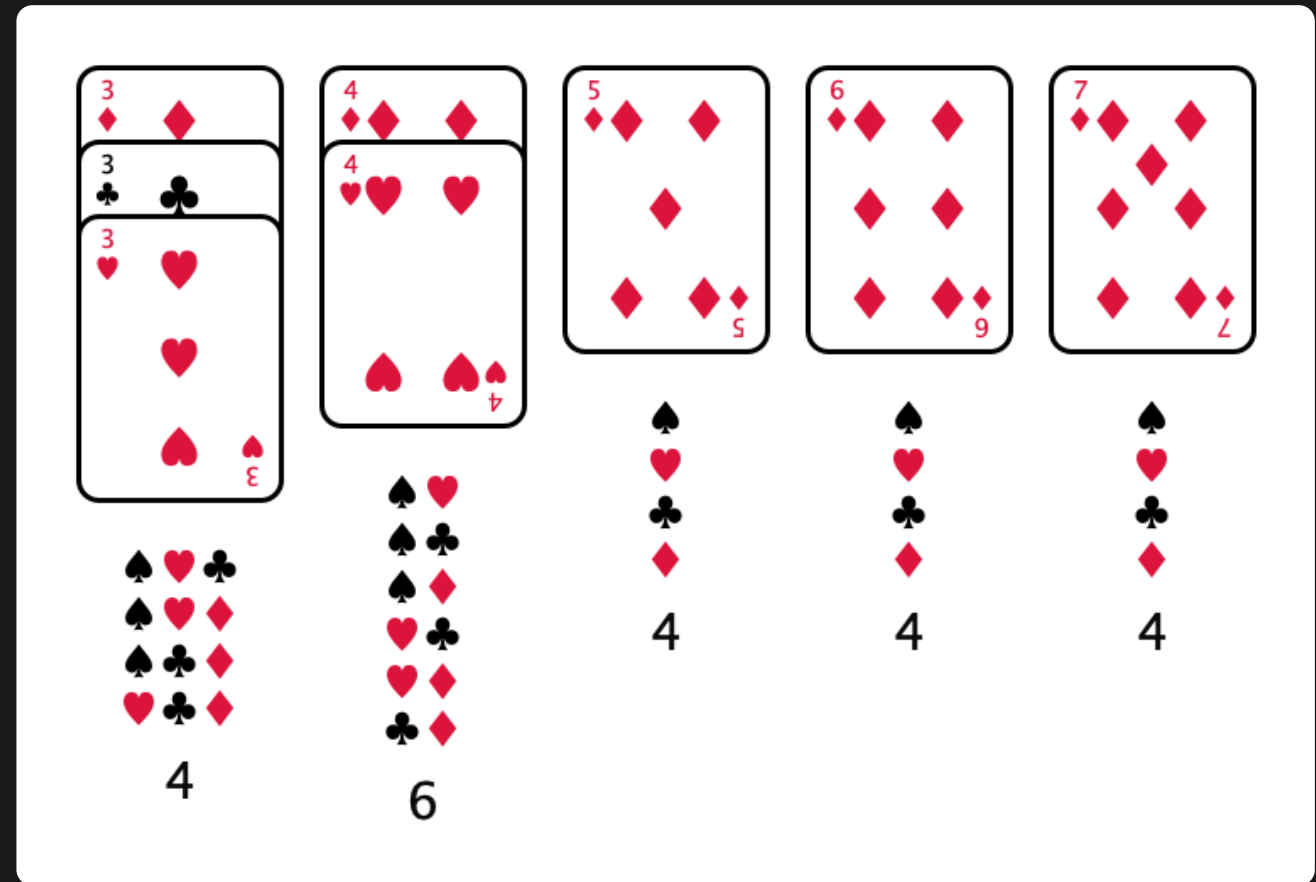
The Suit Factor

$$\{3^1, 2^1, 1^3\}$$

Under each rank, every way to assign the suits

card count	multiplicity	suit factor
3	1	$\binom{4}{3} = 4$
2	1	$\binom{4}{2} = 6$
1	3	$\binom{4}{1}^3 = 64$

$$\begin{aligned}
 Su(d) &= \binom{4}{3} \binom{4}{2} \binom{4}{1}^3 \\
 &= 4 \times 6 \times 64 = 1,536
 \end{aligned}$$



A General Degeneracy Class

$\{3, 2, 1^3\}$: card counts $a_1, a_2, a_3 = 3, 2, 1$; multiplicities $m(3) = 1, m(2) = 1, m(1) = 3; t = 3; r = 5$

$$\{a_1^{m(a_1)}, a_2^{m(a_2)}, \dots, a_t^{m(a_t)}\}$$

a_i how many cards of a rank, between 1 and 4; t distinct values

$m(a_i)$ a function that gives the multiplicity of each element: how many ranks have a_i cards

$r = \sum_{i=1}^t m(a_i)$ the number of distinct ranks

The Formulae

$$Su(\{a_1^{m(a_1)}, \dots, a_t^{m(a_t)}\}) = \prod_{i=1}^t \binom{4}{a_i}^{m(a_i)}$$

$$\begin{aligned} P(\{a_1^{m(a_1)}, a_2^{m(a_2)}, \dots, a_t^{m(a_t)}\}) &= \binom{r}{m(a_1), m(a_2), \dots, m(a_t)} \\ &= \frac{r!}{m(a_1)!m(a_2)! \cdots m(a_t)!} \end{aligned}$$

Eight Cards

class	multiset	r	$S(r)$	$P(d)$	$Su(d)$
no duplicates	$\{1^8\}$	8	?	$\binom{8}{8} = 1$	$\binom{4}{1}^8 = 65,536$
one pair	$\{2^1, 1^6\}$	7	?	$\binom{7}{1,6} = 7$	$\binom{4}{2} \binom{4}{1}^6 = 24,576$
two pairs	$\{2^2, 1^4\}$	6	71	$\binom{6}{2,4} = 15$	$\binom{4}{2}^2 \binom{4}{1}^4 = 9,216$
three of a kind	$\{3^1, 1^5\}$	6	71	$\binom{6}{1,5} = 6$	$\binom{4}{3} \binom{4}{1}^5 = 4,096$
three pairs	$\{2^3, 1^2\}$	5	10	$\binom{5}{3,2} = 10$	$\binom{4}{2}^3 \binom{4}{1}^2 = 3,456$
full house	$\{3^1, 2^1, 1^3\}$	5	10	$\binom{5}{1,1,3} = 20$	$\binom{4}{3} \binom{4}{2} \binom{4}{1}^3 = 1,536$
four of a kind	$\{4^1, 1^4\}$	5	10	$\binom{5}{1,4} = 5$	$\binom{4}{4} \binom{4}{1}^4 = 256$

We have a "nice" formula for two columns but not for S sadly.



The "Algorithm" in Scala

$$\text{hands with a straight} = \sum_d S(r) P(d) Su(d)$$

```
1 def computeStraightCount(handSize: Int): Long =
2   getDegeneracyClasses(handSize)
3   .map { d =>
4     countStraightRankSets(d) * placementFactor(d) * suitFactor(d)
5   }
6   .sum
7
8 def getDegeneracyClasses(handSize: Int): List[DegeneracyClass] = ???
9 def countStraightRankSets(d: DegeneracyClass): Long = ???
10 def placementFactor(d: DegeneracyClass): Long = ???
11 def suitFactor(d: DegeneracyClass): Long = ???
```

Integer Partitions of 8

An **integer partition** of 8 writes 8 as a sum of positive whole numbers, order ignored

class	partition of 8	multiset	r
no duplicates	$1+1+1+1+1+1+1+1$	$\{1^8\}$	8
one pair	$2+1+1+1+1+1+1$	$\{2^1, 1^6\}$	7
two pairs	$2+2+1+1+1+1$	$\{2^2, 1^4\}$	6
three of a kind	$3+1+1+1+1+1$	$\{3^1, 1^5\}$	6
three pairs	$2+2+2+1+1$	$\{2^3, 1^2\}$	5
full house	$3+2+1+1+1$	$\{3^1, 2^1, 1^3\}$	5
four of a kind	$4+1+1+1+1$	$\{4^1, 1^4\}$	5

There are actually 22 partitions of 8 but only 7 of them are relevant to us.

Parts of at most 4, because there are 4 suits. At least 5 parts, because a straight needs 5 ranks

Degeneracy Classes, in Scala

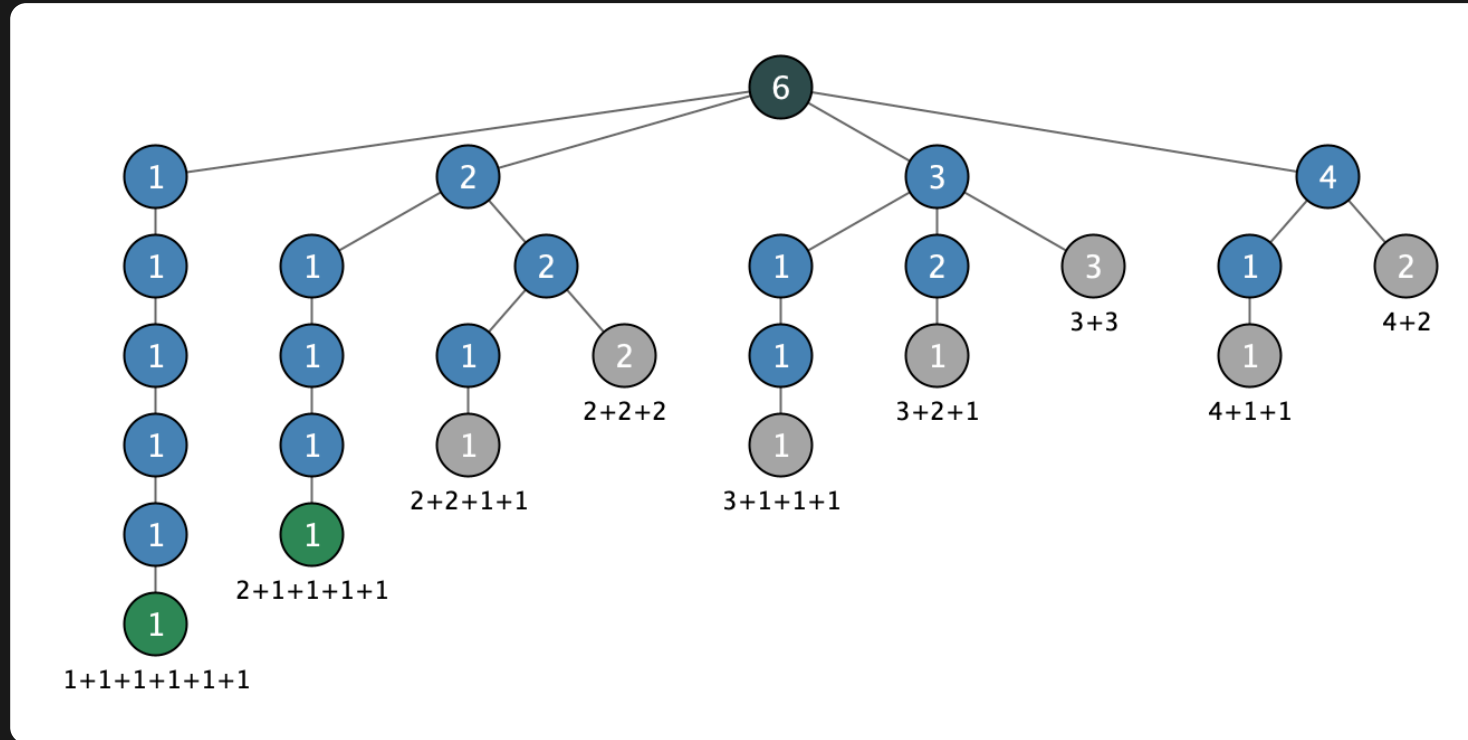
```
1 type CardCount = Int :| Interval.Closed[1, 4]
2 type Frequency = Int :| Positive
3
4 type DegeneracyClass = Map[CardCount, Frequency]
```

$\{2^2, 1^4\}$ is `Map(2 -> 2, 1 -> 4)` : card count to multiplicity

`:|` is iron: a refined `Int` the compiler knows is in range

```
1 def integerPartitions(n: Int, maxPart: Int): List[List[Int]] =
2   if n == 0 then List(Nil)
3   else
4     for
5       part <- (1 to math.min(n, maxPart)).toList
6       tail <- integerPartitions(n - part, part)
7     yield part :: tail
8
9 def getDegeneracyClasses(handSize: Int): List[DegeneracyClass] =
10   integerPartitions(handSize, maxPart = 4)
11   .filter(_.length >= 5)
12   .map(DegeneracyClass.fromParts)
```

Partitions of 6, Recursively



Each branch picks the next part, never bigger than the one before; `maxPart = 4` rules out 5 and 6

● at least 5 parts, kept ● dropped by the filter

The two survivors are the two six-card templates: $\{1^6\}$ and $\{2, 1^4\}$

From Parts to a Multiset

```
1 object DegeneracyClass:
2   def fromParts(parts: List[Int]): DegeneracyClass =
3     toMultiset(parts.map(_.refineUnsafe[Interval.Closed[1, 4]]))
4
5   private def toMultiset(parts: List[CardCount]): DegeneracyClass =
6     parts
7       .groupMapReduce(identity)(Function.const(1: Frequency))(addFrequencies)
8
9   private def addFrequencies(x: Frequency, y: Frequency): Frequency =
10    (x + y).assume[Positive]
```

`List(2, 2, 1, 1, 1, 1)` becomes `Map(2 -> 2, 1 -> 4)`

`groupMapReduce`: group equal parts, map each to 1, add up the 1s

Eight Cards, Seven Classes

```
found 7 degen classes for hand size of 8:  
  {18}  
  {2, 16}  
  {22, 14}  
  {23, 12}  
  {3, 15}  
  {3, 2, 13}  
  {4, 14}
```

This is what my **Show** instance (cats' type class for printing) spits out

Placements, in Scala

$$P(\{a_1^{m(a_1)}, a_2^{m(a_2)} \dots, a_t^{m(a_t)}\}) = \frac{r!}{m(a_1)!m(a_2)! \dots m(a_t)!}$$

```
1 def placementFactor(d: DegeneracyClass): Long =  
2   multinomial(d.values.toList)  
3  
4 def multinomial(ks: List[Int]): Long =  
5   (factorial(ks.sum) / ks.map(factorial).product).toLong  
6  
7 def factorial(n: Int): BigInt =  
8   (1 to n).map(BigInt.apply).product
```

`d.values` are the multiplicities $m(a_i)$, and they add up to r

The Suit Factor, in Scala

$$Su(\{a_1^{m(a_1)}, \dots, a_t^{m(a_t)}\}) = \prod_{i=1}^t \binom{4}{a_i}^{m(a_i)}$$

```
1 def suitFactor(d: DegeneracyClass): Long =
2   d.toList
3     .map((cardCount, frequency) => binomial(4, cardCount) `pow` frequency)
4     .product
5
6 def binomial(n: Int, k: Int): Long =
7   (factorial(n) / factorial(n - k) / factorial(k)).toLong
8
9 extension (l: Long)
10   def pow(exponent: Int): Long =
11     BigInt(l).pow(exponent).toLong
```

Rank Sets With a Straight

```
1 type RankSet = Set[Rank]
2
3 def countStraightRankSets(d: DegeneracyClass): Long =
4     rankSetsWithStraight(d.values.sum).size
5
6 def rankSetsWithStraight(r: Int): Set[RankSet] =
7     if r < 5 then Set.empty
8     else if r == 5 then allFiveRankStraights
9     else
10         for
11             rankSet <- rankSetsWithStraight(r - 1)
12             otherRank <- Rank.values.toSet diff rankSet
13         yield rankSet + otherRank
```

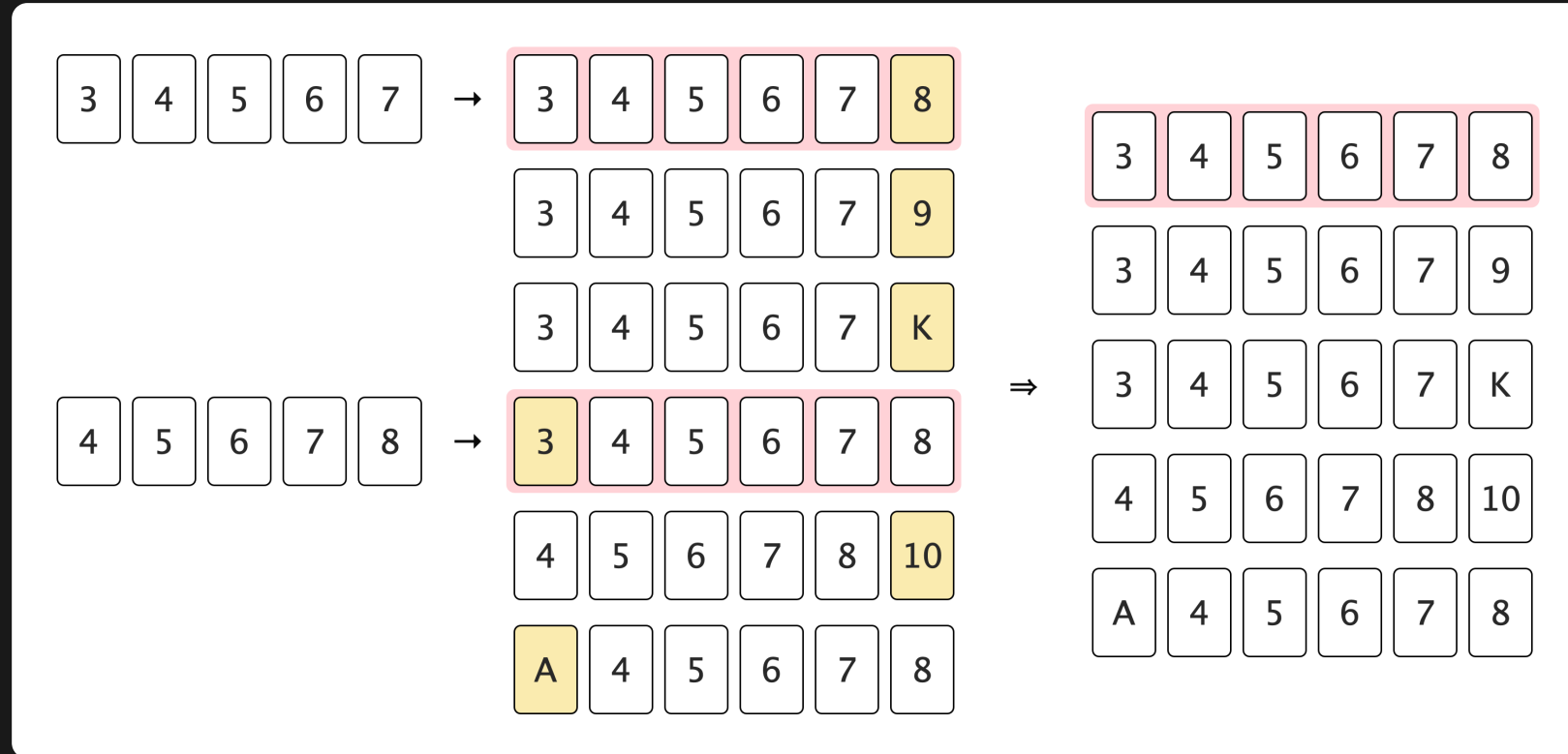
Base case: the ten straights (`allFiveRankStraights` , next slide). Step: add any rank the set doesn't have; still a straight. Repeat.

The Ten Straights

```
1 lazy val allFiveRankStraights: Set[RankSet] =  
2   Iterator.iterate(Rank.Ace)(_nextRank).take(10).map(mkFiveCardStraight).toSet  
3  
4 def mkFiveCardStraight(start: Rank): RankSet =  
5   Iterator.iterate(start)(_nextRank).take(5).toSet  
6  
7 extension (r: Rank)  
8   def nextRank: Rank = Rank.fromOrdinal((r.ordinal + 1) % 13)
```

Ten starting ranks, Ace to Ten. The `% 13` wraps King round to Ace, so Ten gives T J Q K A. Nothing starts above Ten, so nothing wraps any further: no Q K A 2 3.

Rank Sets With a Straight: One Step



■ the added rank ■ built twice, but deduped because I'm using **Set**

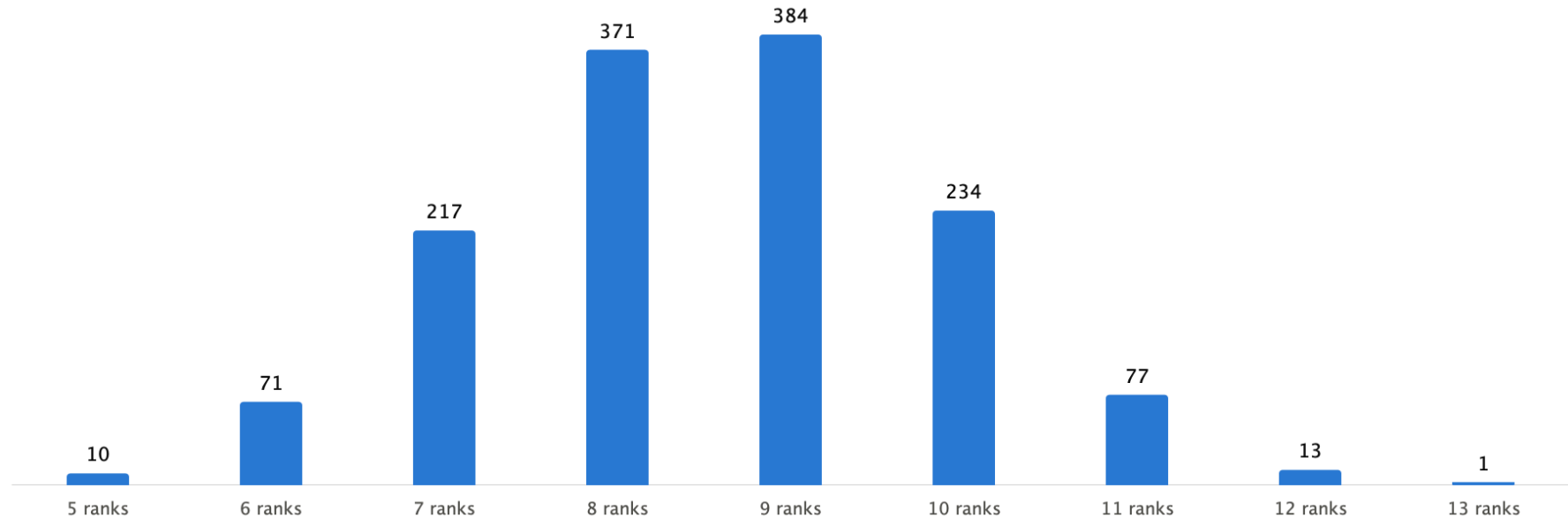
Every straight, plus every rank it doesn't have: $10 \times 8 = 80$ rank sets

Each of the 9 runs of six is built twice, so the **Set** keeps $80 - 9 = 71 = S(6)$

Rank Sets With a Straight, Counted

```
(5 to 13).map(r => r -> rankSetsWithStraight(r).size).foreach(println)
```

```
(5,10)  
(6,71)  
(7,217)  
(8,371)  
(9,384)  
(10,234)  
(11,77)  
(12,13)  
(13,1)
```



10 and 71 match what we counted by hand. Not in the OEIS (the On-Line Encyclopedia of Integer Sequences), sadly.

Eight Cards, Again

class	multiset	r	$S(r)$	$P(d)$	$Su(d)$	product
no duplicates	$\{1^8\}$	8	371	$\binom{8}{8} = 1$	$\binom{4}{1}^8 = 65,536$	24,313,856
one pair	$\{2^1, 1^6\}$	7	217	$\binom{7}{1,6} = 7$	$\binom{4}{2} \binom{4}{1}^6 = 24,576$	37,330,944
two pairs	$\{2^2, 1^4\}$	6	71	$\binom{6}{2,4} = 15$	$\binom{4}{2}^2 \binom{4}{1}^4 = 9,216$	9,815,040
three of a kind	$\{3^1, 1^5\}$	6	71	$\binom{6}{1,5} = 6$	$\binom{4}{3} \binom{4}{1}^5 = 4,096$	1,744,896
three pairs	$\{2^3, 1^2\}$	5	10	$\binom{5}{3,2} = 10$	$\binom{4}{2}^3 \binom{4}{1}^2 = 3,456$	345,600
full house	$\{3^1, 2^1, 1^3\}$	5	10	$\binom{5}{1,1,3} = 20$	$\binom{4}{3} \binom{4}{2} \binom{4}{1}^3 = 1,536$	307,200
four of a kind	$\{4^1, 1^4\}$	5	10	$\binom{5}{1,4} = 5$	$\binom{4}{4} \binom{4}{1}^4 = 256$	12,800
total: 9.82%						73,870,336

Run It

```
(5 to 13).map(n => n -> computeStraightCount(n))
```

Hand size	Total hands	With a straight	Probability
5	2,598,960	10,240	0.39%
6	20,358,520	367,616	1.81%
7	133,784,560	6,454,272	4.82%
8	752,538,150	73,870,336	9.82%
9	3,679,075,400	619,588,736	16.84%
10	15,820,024,220	4,051,217,344	25.61%
11	60,403,728,840	21,461,806,976	35.53%
12	206,379,406,870	94,674,009,184	45.87%
13	635,013,559,600	355,161,047,872	55.93%

Now With Gaps

$$\sum_d S_g(r) P(d) Su(d)$$

Only S knows what a straight is, so only S changes: $S(r)$ becomes $S_g(r)$, with g the biggest gap allowed

Normal straights correspond to S_0 . The Shortcut Joker corresponds to S_1 . $P(d)$ and $Su(d)$ remain the same

```
1 def computeStraightCount(handSize: Int, maxGap: Int): Long =  
2   getDegeneracyClasses(handSize)  
3     .map { d =>  
4       countStraightRankSets(d, maxGap) * placementFactor(d) * suitFactor(d)  
5     }  
6     .sum
```

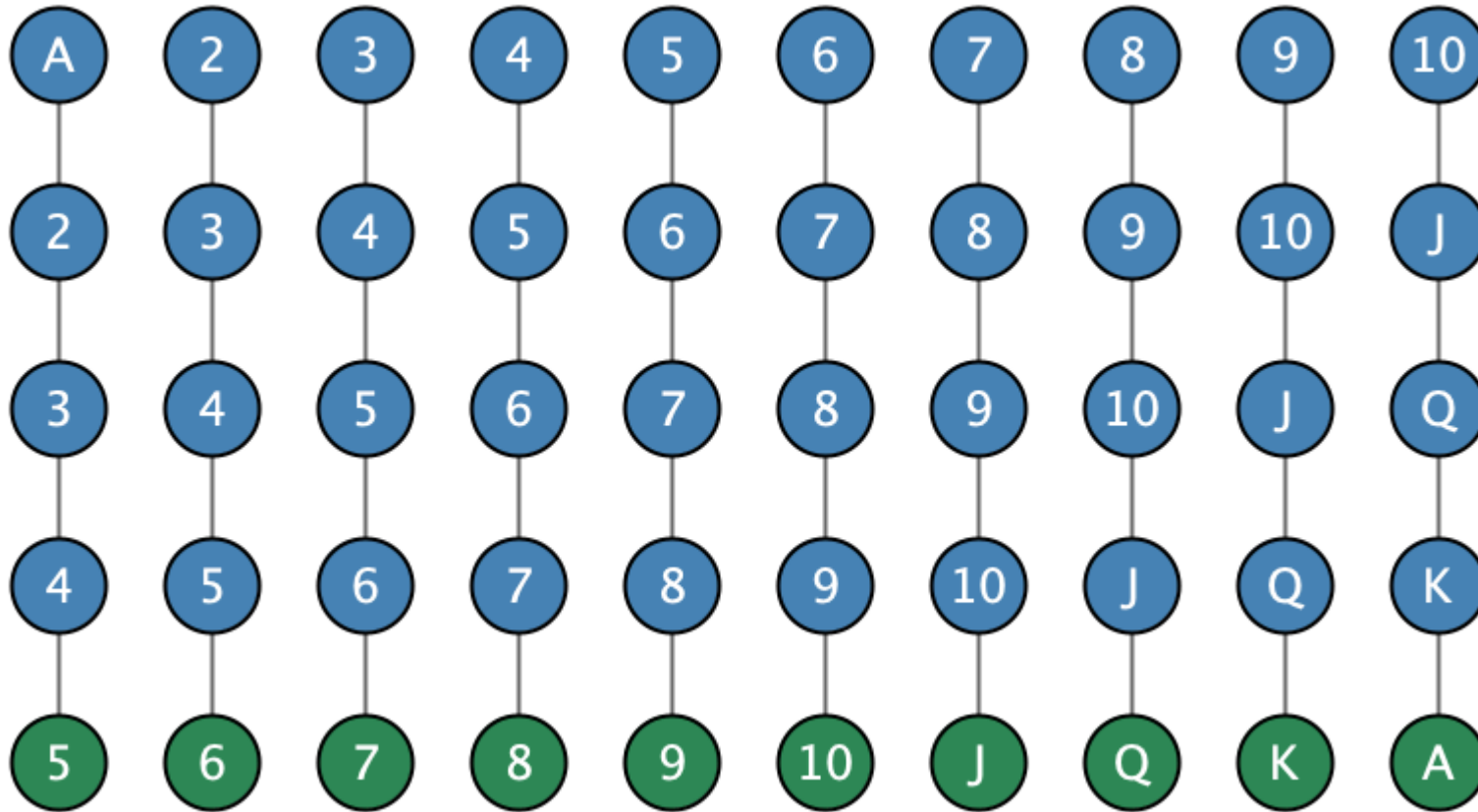
The recursion for S still works, so only its base case changes

Threading the Gap Through

```
1 def countStraightRankSets(d: DegeneracyClass, maxGap: Int): Long =
2   rankSetsWithStraight(d.values.sum, maxGap).size
3
4 def rankSetsWithStraight(r: Int, maxGap: Int): Set[RankSet] =
5   if r < 5 then Set.empty
6   else if r == 5 then allFiveRankStraights(maxGap)
7   else
8     for
9       rankSet    <- rankSetsWithStraight(r - 1, maxGap)
10      otherRank <- Rank.values.toSet diff rankSet
11    yield rankSet + otherRank
```

Only the base case changes: `allFiveRankStraights(maxGap)` builds a tree of straights from every starting rank

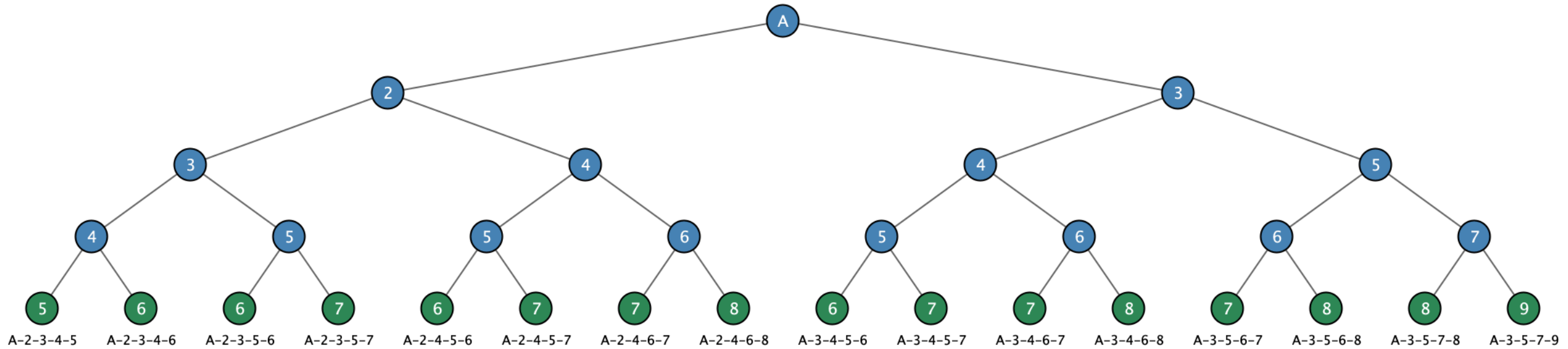
Base Case for Normal Straights



Start with the Ace low straight

And then generate the rest by increasing all ranks by one

Base Case for Gappy Straights



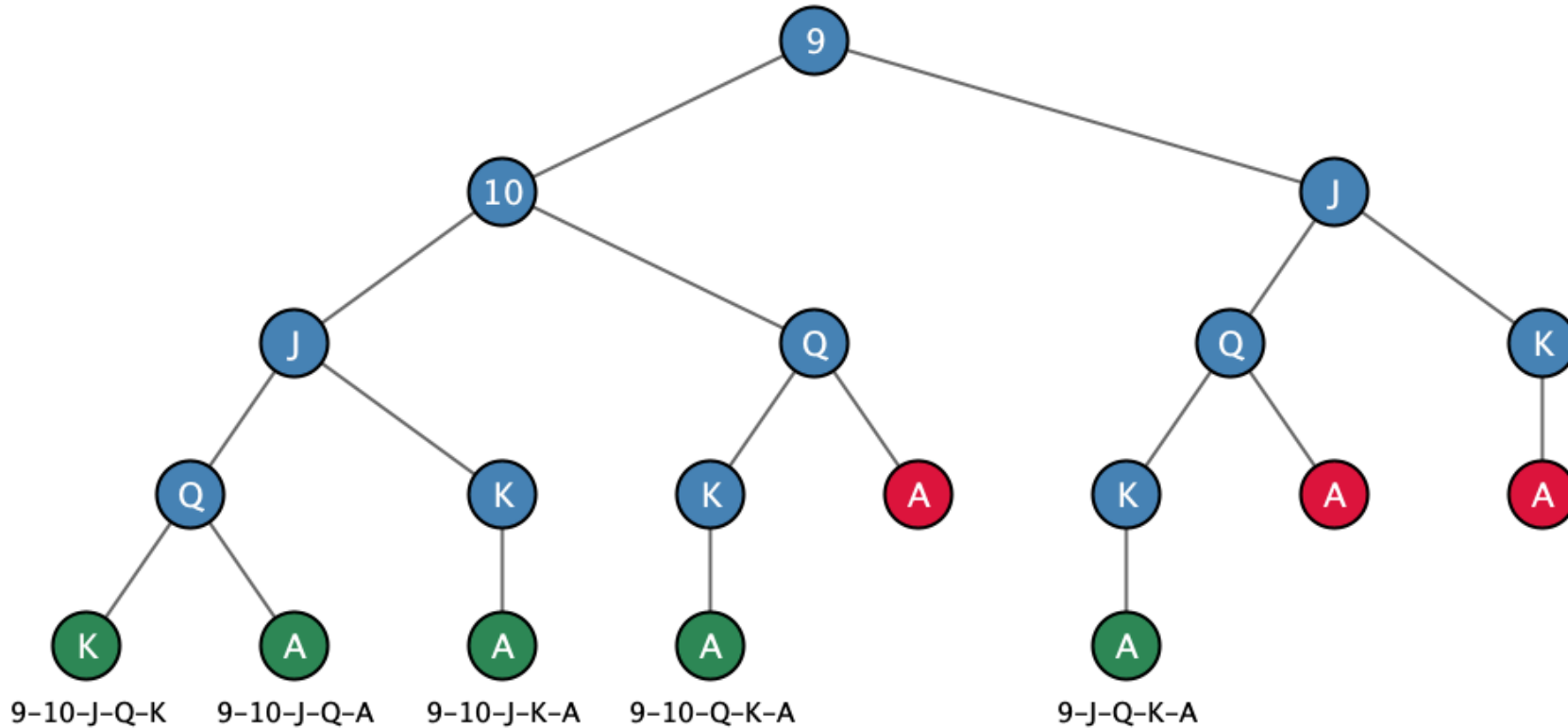
From an Ace the next rank can be a 2 *or* a 3! Each time we select the next rank there are actually 2 choices

$$2^4 = 16 \text{ Ace-low gappy straights}$$

Base Case for Gappy Straights, in Scala

```
1 def genStraightsWithAceLow(maxGap: Int): List[RankSet] =
2   def go(state: List[RankSet], size: Int): List[RankSet] =
3     if size == 5 then state
4     else
5       go(
6         for
7           hand    <- state
8           step    <- (1 to maxGap + 1).toList
9           newRank = hand.map(_.ordinal).max + step
10          if newRank <= Rank.King.ordinal
11          yield hand + Rank.fromOrdinal(newRank),
12            size + 1,
13        )
14   go(List(Set(Rank.Ace)), 1)
```

Start With Cards Other Than Ace



The same tree from every start

- a gappy straight
- past the Ace, pruned

128 gappy straights,
instead of 10

Fewer than the $16 \times 10 = 160$ the tree suggests: trees that start high get pruned at the Ace

Every Starting Rank, in Scala

```
1 val aceHighOrdinal = Rank.King.ordinal + 1
2
3 private def allFiveRankStraights(maxGap: Int): Set[RankSet] = {
4     val aceLowAsOrdinals = genStraightsWithAceLow(maxGap).map(_.map(_.ordinal))
5
6     Iterator
7         .iterate(aceLowAsOrdinals)(
8             _.map(_.map(_ + 1)).filter(rs => !rs.contains(aceHighOrdinal + 1))
9         )
10    .takeWhile(_.nonEmpty)
11    .flatten
12    .map(_.map(x =>
13        if x == aceHighOrdinal then Rank.Ace else Rank.fromOrdinal(x)
14    ))
15    .toSet
16 }
```

Add 1 to everything each round, kicking out anything past the high Ace, until nothing is left

Then flatten, and map the ordinals back to ranks: 128 gappy straights

Eight Cards, With Gaps

class	multiset	r	$S_1(r)$	$P(d)$	$Su(d)$	product
no duplicates	$\{1^8\}$	8	1,156	$\binom{8}{8} = 1$	$\binom{4}{1}^8 = 65,536$	75,759,616
one pair	$\{2^1, 1^6\}$	7	1,096	$\binom{7}{1,6} = 7$	$\binom{4}{2} \binom{4}{1}^6 = 24,576$	188,547,072
two pairs	$\{2^2, 1^4\}$	6	576	$\binom{6}{2,4} = 15$	$\binom{4}{2}^2 \binom{4}{1}^4 = 9,216$	79,626,240
three of a kind	$\{3^1, 1^5\}$	6	576	$\binom{6}{1,5} = 6$	$\binom{4}{3} \binom{4}{1}^5 = 4,096$	14,155,776
three pairs	$\{2^3, 1^2\}$	5	128	$\binom{5}{3,2} = 10$	$\binom{4}{2}^3 \binom{4}{1}^2 = 3,456$	4,423,680
full house	$\{3^1, 2^1, 1^3\}$	5	128	$\binom{5}{1,1,3} = 20$	$\binom{4}{3} \binom{4}{2} \binom{4}{1}^3 = 1,536$	3,932,160
four of a kind	$\{4^1, 1^4\}$	5	128	$\binom{5}{1,4} = 5$	$\binom{4}{4} \binom{4}{1}^4 = 256$	163,840
total						366,608,384

Rank Sets With a Gappy Straight

r	$S_0(r)$	$S_1(r)$
5	10	128
6	71	576
7	217	1,096
8	371	1,156
9	384	709
10	234	286
11	77	78
12	13	13
13	1	1

The recursive step is the same, it's just that with gappy straights we have more to begin with!

The Answer

$$\frac{366,608,384}{752,538,150} = 48.72\%$$



Without the joker it was about 10%, so having this joker makes getting a straight 5 times easier!!!

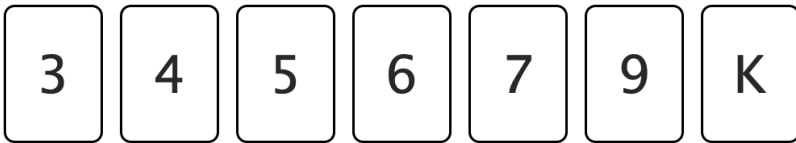
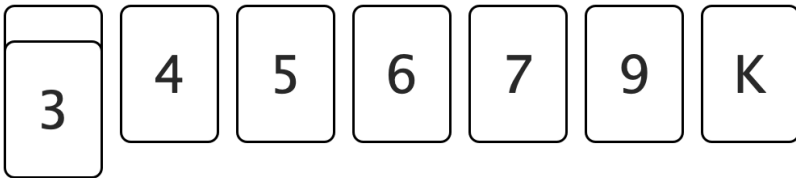
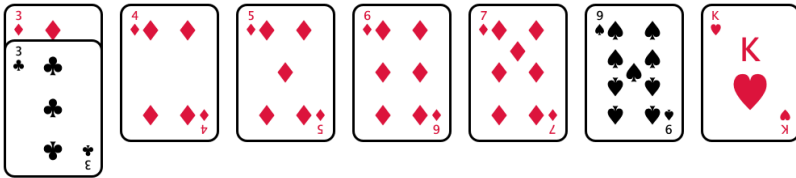
All of It

Hand size	Gap 0	Gap 1	Gap 2	Gap 3	Gap 4
5	0.39%	5.04%	18.56%	33.69%	43.26%
6	1.81%	16.42%	41.95%	62.84%	74.68%
7	4.82%	31.99%	62.07%	80.36%	89.56%
8	9.82%	48.72%	76.83%	89.87%	95.71%
9	16.84%	63.94%	86.66%	94.89%	98.20%
10	25.61%	76.17%	92.73%	97.50%	99.23%
11	35.53%	85.07%	96.23%	98.82%	99.68%
12	45.87%	91.06%	98.13%	99.46%	99.87%
13	55.93%	94.86%	99.11%	99.76%	99.95%

Gap 0 is a normal straight. Gap 1 is the Shortcut Joker. Gaps 2 to 4 need a joker that doesn't exist yet

One Idea, Three Ways

Whether a hand contains a straight depends only on its rank set



$$\sum_d S_g(r) P(d) Su(d)$$

```
getDegeneracyClasses(handSize)
  .map { d =>
    countStraightRankSets(d, maxGap) * placementFactor(d) * suitFactor(d)
  }
  .sum
```

An Exercise for the Reader



Four Fingers

All Flushes and Straights can be made with 4 cards

Which parts of $\sum_d S(r) P(d) Su(d)$ change?

What are the odds of a straight in 8 cards?
And with the Shortcut Joker as well?

Thanks

Noel Welsh, for mentoring me

Doodle, for the cards, trees and charts

The Math Stack Exchange answer on straights in Texas Hold'em, for the degeneracy classes

LocalThunk and Playstack, for Balatro: the screenshot and the joker art

The blog post: stanosphere.github.io/blog/counting-balatro-straights